



PDF Download
3674829.3675059.pdf
01 January 2026
Total Citations: 0
Total Downloads: 212

 Latest updates: <https://dl.acm.org/doi/10.1145/3674829.3675059>

SHORT-PAPER

Mapping Thermal Footprints: Occupancy Estimation and Localization in Diverse Indoor Settings with Thermal Arrays

SOUMYA RANJAN SAHOO, Indian Institute of Science Education and Research Bhopal, Bhopal, MP, India

HAROON R LONE, Indian Institute of Science Education and Research Bhopal, Bhopal, MP, India

Open Access Support provided by:

Indian Institute of Science Education and Research Bhopal

Published: 08 July 2024

[Citation in BibTeX format](#)

COMPASS '24: ACM SIGCAS/SIGCHI
Conference on Computing and
Sustainable Societies
July 8 - 11, 2024
New Delhi, India

Conference Sponsors:
SIGCAS
SIGCHI

Mapping Thermal Footprints: Occupancy Estimation and Localization in Diverse Indoor Settings with Thermal Arrays

Soumya Ranjan Sahoo
IISER Bhopal
Bhopal, India
soumya18@iiserb.ac.in

Haroon R Lone
IISER Bhopal
Bhopal, India
haroon@iiserb.ac.in

ABSTRACT

Estimating and locating occupants indoors is crucial for automating operations within buildings. However, current privacy-preserving occupancy estimation systems using thermal cameras have not been thoroughly evaluated in dense settings, such as classrooms or movie theaters, where occupants sit close to one another. Estimating occupancy in such settings presents challenges, as nearby occupants often lead to clusters of thermal signatures, thereby affecting the accuracy of the estimation. In response to these challenges, our work proposes Machine Learning (ML) and Deep Learning (DL) based methods for occupancy estimation and localization, both in dense and sparse settings. The ML method complements existing occupancy estimation techniques by incorporating new manual features, while the DL method performs automatic feature extraction, accurate occupancy estimation and localization. Remarkably, the results demonstrate a significant enhancement in occupancy estimation accuracy of up to 10%, achieving an impressive overall accuracy rate of 97%. Moreover, our evaluation on an edge device confirms the practicality and relevance of the proposed methods in real-world applications.

CCS CONCEPTS

• **Computing methodologies** → **Machine learning; Supervised learning by classification; Feature selection; Neural networks; Computer vision**; • **Computer systems organization** → **Sensors and actuators; Signal processing systems**; • **Hardware** → **Sensor applications and deployments**; • **Human-centered computing** → **Ubiquitous computing applications; Ubiquitous computing theories and models**.

KEYWORDS

Occupancy estimation, localization, thermal images, MLX90640

ACM Reference Format:

Soumya Ranjan Sahoo and Haroon R Lone. 2024. Mapping Thermal Footprints: Occupancy Estimation and Localization in Diverse Indoor Settings with Thermal Arrays. In *ACM SIGCAS/SIGCHI Conference on Computing and Sustainable Societies (COMPASS '24)*, July 08–11, 2024, New Delhi, India. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3674829.3675059>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

COMPASS '24, July 08–11, 2024, New Delhi, India

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1048-3/24/07

<https://doi.org/10.1145/3674829.3675059>

1 INTRODUCTION

Occupancy *estimation* is essential in buildings to optimize space management, safety and security, and facility maintenance such as HVAC (Heating, Ventilation, Air-Conditioning) and lighting systems. Research shows that buildings account for up to 30% of total energy usage [26]. With automated occupancy estimation systems, the occupancy data can be fed into the HVAC system for managing and optimizing the energy consumption of the building's electrical appliances [12]. However, existing frameworks leveraging different sensing techniques to estimate the occupancy suffer from low accuracy or are privacy averse [14, 24, 35]. To address these challenges, thermal array sensors measuring the thermal radiation emitted by occupants offer a promising solution.

Utilizing thermal cameras for occupancy estimation presents dual advantages in utility and cost-effectiveness. Unlike regular cameras, thermal cameras detect infrared radiation, ensuring reliable detection in various lighting conditions and addressing privacy concerns. Their 24/7 operational capability, resilience to environmental factors, and reduced maintenance contribute to enhanced utility and cost-effectiveness. While initial costs may have been higher historically, technological advancements have made thermal cameras more affordable. The non-intrusive nature of thermal imaging aligns with privacy considerations, which make them socially acceptable.

Previous research on occupancy estimation using thermal cameras has been limited to sparse settings, such as round tables or cubicles, where the maximum number of occupants were typically five to six [8, 21, 33]. However, in real-world scenarios, we often encounter dense settings where occupants sit close to each other, such as in classrooms or movie theaters. Estimating occupancy in dense settings presents challenges because the heat signatures emitted by neighboring occupants often form clusters, making accurate occupancy estimation difficult. Furthermore, environmental variability, influenced by factors like temperature changes, can also impact model accuracy across diverse conditions. Recently, Sahoo et al. [25] conducted a study in dense settings and detected up to 11 occupants in a group of 15 with an accuracy of 91%. This result highlights the need to improve the accuracy of occupancy estimation in dense settings.

Furthermore, *localizing* occupants indoors is also crucial to optimize energy efficiency and comfort by applying zone-based control [6, 30]. Such an approach optimizes energy consumption by selectively activating and controlling specific areas only where occupants are present. In smart building scenarios, *localization* can enhance the overall occupant experience by providing location-based services and personalization.

In this work, we leverage thermal signatures and propose Machine Learning (ML) and Deep Learning (DL) based methods for occupancy estimation and localization. The proposed ML method incorporates novel manual textural features from the computer vision domain by extracting two-dimensional spatio-temporal features from thermal images. However, the proposed DL-based method does automatic feature extraction and performs occupancy estimation and localization.

In the ML-based method, we first compute features manually and later use Random Forest for occupancy estimation. In contrast, we use YOLO [15] for occupancy estimation and localization as the DL-based method. We evaluate these methods using our unique dataset, which was collected using a low-cost, low-resolution thermal camera, MLX90640. This dataset encompasses both static and dynamic scenarios in sparse and dense settings. In dynamic scenarios, occupants are in motion, whereas in static scenarios, occupants remain stationary. Dynamic scenarios present a complex evaluation process as the moving occupants increase clustered heat signatures (occlusion) and noise within the thermal images. Moreover, we assess the robustness of these methods using a publicly available dataset.

Additionally, we deployed our models on an edge device, Raspberry Pi, to assess their real-time performance in terms of latency, accuracy, etc. Our results showed that our models improved existing methods substantially and are energy efficient. The major contribution of our work includes:

- We have proposed a unique feature vector set required for ML-based occupancy estimation. The proposed features address the limitations of existing occupancy estimation methods and offer a comprehensive solution.
- We have utilized a DL-based method (i.e., YOLO) for both occupancy estimation and localization. The beauty of this approach lies in its automatic feature extraction and simultaneous occupancy estimation and localization.
- We assessed our methods using two distinct datasets: INFRADEL and Thermo-Presence. INFRADEL, our proprietary dataset, is diverse, encompassing both sparse and dense settings. In this paper, we make INFRADEL publicly available to foster further research in this domain [31]. Conversely, Thermo-Presence is an openly accessible dataset.
- We compared the performance of the proposed methods with a baseline method (i.e., U-Net) for occupancy estimation.
- We have deployed our models on an edge device, Raspberry Pi, for real-time inference of thermal images, which proves the models' usability in real-world scenarios. Furthermore, we compared the performance of the models with two other baselines on the edge device.

2 RELATED WORKS

This section briefly overviews prior research on occupancy estimation and localization in indoor environments. Existing occupancy estimation methods either use RGB images or multiple sensor frameworks such as PIR [2], ultrasonic [28], radio frequency (RF) [27], Wi-Fi [20, 29, 36], CO₂ [16], LED based [18, 34], and sensor fusion [3, 11, 22].

Occupancy Estimation: Advances in thermal sensor technology improved detection accuracy, reduced cost, and offered flexible deployment options [1, 23]. Thermal sensors are preferred for occupancy estimation as they preserve privacy by collecting heat signatures of human heads, referred to as “blobs” in the computer vision field. Usually, occupancy estimation relies on blob detection techniques, where the number of blobs corresponds to the number of occupants in settings with low occupancies, such as round tables or cubicle settings [5]. For example, Tyndall et al. [32] proposed an occupancy estimation sensor system based on a 4×16 thermal detector array, incorporating machine learning classifiers to accurately determine the number of occupants within the sensor's field of view. Their evaluation found that nominal classifiers exhibit enhanced robustness compared to numerical classifiers. Meanwhile, Beltran et al. introduced ThermoSense [6], a novel system that estimates occupancy using K-means, Linear regression, and Artificial Neural Network based architectures. Lastly, Cokbas et al. [10] proposed a people counting system that utilizes a low-resolution thermal sensor in an overhead tripwire configuration, offering accurate detection and tracking of transient entries and exits at entryways. However, it becomes challenging to estimate occupancy accurately when people sit very close to each other or when they are in motion, as shown in Figure 1. Such settings often result in the clustering of heat signatures or blobs. To address this complexity, Chidurala et al. [8] introduced Blob Filtering Algorithm (BFA) using threshold-based manual features. Later, by adding adaptive features, Sahoo et al. [25] improved the BFA with an Adaptive Blob Filtering Algorithm (ABFA). The ABFA effectively distinguishes between individual blobs and clustered blobs.

In addition to ML-based methods, DL models have emerged as highly significant contributors. Metwaly et al. [19] conducted a comprehensive study, evaluating various models like Feedforward, Convolutional, and Recurrent Neural Networks (FNN, CNN, and RNN) on different embedded processors, achieving an impressive 99% prediction accuracy for people counting with minimal RAM usage (2 KB) and low latency (14 ms). Likewise, Kraft et al. [17] introduced the U-Net CNN for occupancy-based studies, integrating Gaussian mixture distribution and image segmentation techniques on thermal images to calculate the final occupancy count with promising results. Their work demonstrated the potential of deep learning in enhancing occupancy estimation methods. Furthermore, a notable recent contribution by Xie et al. [33] involved an extensive architectural exploration of six different types of Deep Learning CNN Models such as Single-frame, Multi-Channel, Concatenated, LSTM, majority voting, and TCN. Surprisingly, despite the models' simplicity, they show that these models cannot only achieve

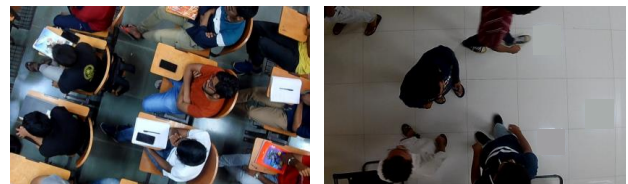


Figure 1: RGB images (ground truth): static dense setting (left), and dynamic setting (right)

remarkably high accuracy compared to state-of-the-art algorithms ([17, 19]), but also obtain benefits in terms of energy consumption, and latency; further highlighting the significance of deep learning in the field of occupancy estimation.

Localization: In the relatively underexplored area of occupant localization, limited research has been done, thus restricting the implementation of zone-based control in indoor areas. However, Bouazizi et al. [7] contributed significantly by introducing a method that employs low-resolution infrared (IR) array sensors and deep learning. This method achieved an impressive 98% accuracy for detecting the presence and location of up to three people indoors using a 32×24 pixel IR array.

Research Gap(s): Though existing research made valuable contributions to occupancy estimation and localization, they were constrained to sparse settings and limited the range of occupants from 0 (minimum) to 6 (maximum) [7, 21, 33]. Further research is needed to explore more diverse and dense indoor environments with higher occupancy ranges while also exploring new localization methods.

3 SENSING AND DATA COLLECTION

Sensing: We collected data with a setup comprising of MLX90640 thermal camera, Raspberry Pi 4B (with 4GB RAM), an RGB camera for ground truth, and a power bank with a 5V and 3A power rating. We selected the Melexis MLX90640 thermal camera due to its exceptional accuracy, widespread availability, and cost-effectiveness. To maintain privacy and optimize thermal data pre-processing, we mounted the entire setup on the ceiling, capturing the heat signature of occupants' heads.

We adjusted the frame rate based on the scenarios (static, dynamic). For static scenarios, we set the frame rate to 2 FPS (frames per second), as this rate minimizes noise in the thermal data. However, dynamic scenarios with occupants movement required a 4 FPS, as the sensor's sequential reading causes a time delay, which is insignificant for still occupants but results in position differences when occupants move within its coverage area.

The MLX90640 camera divide the sensor's array frame into sub-pages 0 and 1 (i.e., a chessboard pattern) where alternate pixels are registered by the sensor for a particular subpage. During the reading process, the sensor sequentially scans these subpages, introducing a minor time delay. This time delay's impact is nearly negligible when tracking stationary objects. However, when occupants move within the sensor's coverage area within a short time interval, faster than the thermal subpage's sampling rate, their positions differ in the two subpages (i.e., subpage 0 and subpage 1). Consequently, this movement results in two distinct heat signatures in the thermal image when the two subpages are overlaid, even though they correspond to a single human presence. Handling such dynamic cases is a complex task and necessitates the incorporation of additional processing steps.

Data Collection: The data collection process spanned nine days at our Institute: seven days for static scenarios and the remaining two days for dynamic scenarios. We chose two locations to capture data: (i) the Classroom, representing a dense occupant environment, and (ii) the Research Lab and Office, emulating a sparse occupant setting. In the dense classroom setting, the thermal camera's field of view (FOV) was limited to $110^\circ \times 75^\circ$, capturing a maximum of

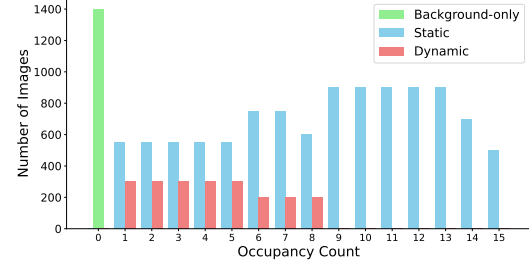


Figure 2: Frequency of images in INFRADEL without data augmentation.

15 occupants. To diversify our data, we collected information from two types of classrooms: one with chairs fixed at the same level and another with chairs fixed at different heights (staircase setting). With the instructor's permission, we incrementally increased the occupancy by adding one occupant at a time to the sensor's coverage area, enabling controlled data collection.

Similarly, we collected data for static and dynamic scenarios in office spaces with cubicle settings. The thermal camera's FOV again captured a maximum of 8 occupants in this setup. The dynamic sparse data in these settings resulted from occupants moving between cubicles, adjusting chair positions, and engaging in activities like raising their hands while using mobile phones or laptops.

We name our collected dataset as INFRADEL (INFRAred Dataset for occupancy Estimation and Localization). In INFRADEL, 50% of the dataset includes occupancy ranging from 1 to 8 covering static & dynamic settings under dense & sparse scenarios. While occupancy levels 9 to 15 exclusively consist of data collected in static dense scenarios, specifically from the classroom setting, contributing to 40% of INFRADEL. Figure 2 shows the frequency distribution of the dataset. Out of the 14,000 collected images, approximately 12,000 belong to static settings, and the rest to dynamic settings. In the figure, occupancy count '0' (remaining 10% of dataset) represents background-only images, i.e., images without an occupant. These images are crucial for one of our proposed methods. We release the dataset publicly with this paper [31]. For more details on the dataset please refer to the Readme file under the cited link.

Data augmentation is crucial in training DL models as it expands the dataset, increases diversity, and prevents overfitting. We used various image augmentation techniques, including scaling, flipping, and adding Gaussian noise to thermal images. These techniques generated three times more data and improved DL-based model's generalization capability. Scaling up and down images captures different object scales, especially in thermal cameras where heat signature sizes for human beings vary with sensor placement height. Horizontal and vertical flipping expands the dataset and reduces orientation bias. Adding Gaussian noise enhances the model's robustness to image noise and variations. The augmented data was only used in deep models (e.g., U-Net, YOLO) as they need diverse data. The data diversification through data augmentation did not help manual feature extraction methods, as augmented images generate the same feature vectors as those in the original thermal image.

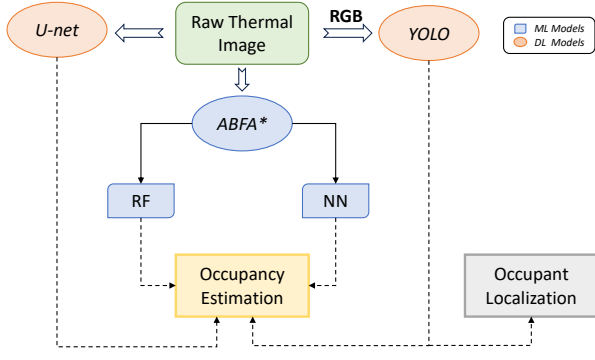


Figure 3: Overview of occupancy estimation and localization methods

4 METHODOLOGY

We propose ML-based method, ABFA* for occupancy estimation and employ a DL-based method, YOLO for both occupancy estimation and localization. Figure 3 provides an overview of our methodology. ABFA* uses machine learning classification algorithms, i.e., Random Forest (RF) and Neural Networks (NN) to estimate the occupancy. Furthermore, we use U-Net as a baseline occupancy estimation model.

Deep learning models, such as YOLO and U-Net, have significantly advanced image analysis tasks, particularly object detection and segmentation. U-Net is a popular choice for semantic image segmentation as it accurately delineates objects within images. On the other hand, YOLO is a state-of-the-art solution for real-time object detection, enabling precise localization and identification of objects in images or video frames. One key advantage of these models is their ability to extract valuable features from raw input data autonomously, making them highly adaptable and flexible for various image types and object categories. Now, we discuss occupancy estimation, localization, and the baseline model in detail.

4.1 Occupancy Estimation

In this sub-section, first we discuss the feature extraction and classification steps of ABFA* in detail. Next, we discuss the occupancy estimation with YOLO.

4.1.1 ABFA*. ABFA* complements existing ABFA [25]. Following is a brief description of the ABFA steps. (i) Noise Reduction - It handles missing pixel values in an image frame by using bilinear interpolation. (ii) Active Pixel and Frame Detection - It decides whether a pixel is active or inactive by comparing it to the corresponding pixel in the threshold frame. A threshold frame is a background frame without any occupant (i.e., frames with 0 occupancy count). Similarly, it detects whether a frame contains any occupant, if it does the frame is marked as active. (iii) Connected Component Analysis - It clusters neighboring active pixels into blobs and outputs the total number of blobs (connected components) in a frame. (iv) Feature Extraction - It extracts features from each of the thermal frames. (v) Classification - It takes extracted features as input and estimates the occupancy count.

ABFA* improves ABFA's accuracy by proposing an *advanced feature vector set* in the 'Feature Extraction' step. Additionally, it integrates the "Sub-frame Discrepancy Detection" step [9] as shown in Figure 4 to handle two distinct heat signatures in the thermal image when the two subpages (i.e., subpage 0 and subpage 1) are overlaid, even though they correspond to a single human presence. The Sub-Frame Discrepancy step focuses on using thermal images only when their individual sub-frames yield an equal number of connected components. This additional step notably enhance the accuracy of dynamic cases. Now, we will first discuss the advanced feature vector set.

Advanced Feature Vector Set of ABFA*: The feature extraction comprises of extracting different features from a thermal image frame, F . The features can be divided into first and second level features in terms of the complexity level [4]. Following is a brief description of these features.

i. First level features: These are one-dimensional histogram features calculated directly from the gray levels of frame F . The image histogram is a two-dimensional representation of how the gray-level pixels are distributed around the image. They are a graphical representation of the optical content (i.e., the amount of light and darkness). These features include (i) Mean (m) (ii), Std. deviation (σ), (iii) Number of active pixels, (iv) Skewness, and (v) Kurtosis.

Mean and standard deviation of the pixel values are key feature vectors as they correlate with the distribution of the pixels within the frame. The more the number of occupants, the higher the pixel values. High pixel values result in a high mean and standard deviation value. *Number of active pixels* is directly proportional to the number of occupants, which makes it a critical feature for classification algorithms. The higher the number of active pixels in the captured frame, the more occupants. Similarly, *Skewness* refers to the degree of asymmetry of a distribution. A distribution is skewed if it is not symmetrical, meaning that one tail of the distribution is longer than the other. *Kurtosis*, on the other hand, measures the degree of peakedness or flatness of a distribution. Both skewness and kurtosis are essential in understanding the distribution characteristics, as they describe the variability of the thermal signatures. We also computed *Number of connected components* by taking the output of the Connected Component Analysis (CCA) block of the ABFA. CCA plays a vital role in identifying and segmenting individual objects within the captured thermal frames.

ii. Second level features: The Gray Level Co-occurrence Matrix (GLCM) introduced by Haralick is one of the oldest operators known for extracting the second level features [13]. These features are higher-order characteristics in terms of complexity as compared

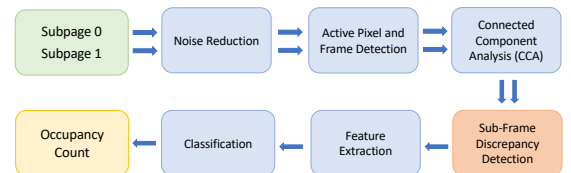


Figure 4: Flowchart of the ABFA* algorithm incorporating Sub-Frame Discrepancy Detection. Subpage 0 and 1 represent input thermal image frames.

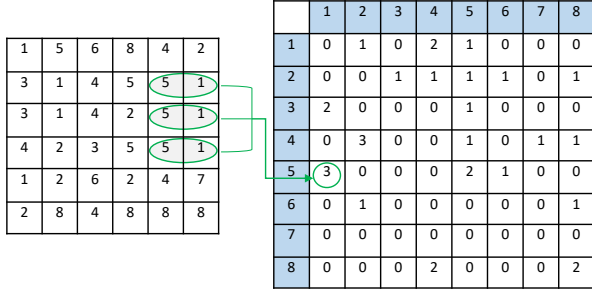


Figure 5: Example showing 8 x 8 GLCM matrix obtained from a 3-bit image with eight brightness levels.

to the first level because they take into account the correlations between image pixels (i.e., the spatial relation between them). To extract texture features from a k -bit image with $L = 2^k$ brightness levels, a square $L \times L$ GLCM is constructed, where each element represents the frequency of occurrence of a pair of pixels with brightness values a and b , separated by d pixels in a certain direction.

Figure 5 show a 3-bit image with $8 (2^3)$ intensity levels, and its 8 x 8 co-occurrence matrix showing the number of occurrences of each pair of adjacent pixels with gray levels (5 1), with a displacement of 1 pixel in the zero-degree direction. In the example, the values of k , L , a , b , and d are 3, 8, 5, 1, and 1, respectively. Typically, the co-occurrence matrix is defined for four main directions: 0, 45, 90, and 135 degrees.

Once the GLCM is calculated, Haralick's second-order statistical features are computed to obtain the textural characteristics of the image. The features include (i) Contrast, (ii) Correlation, (iii) Entropy, (iv) Homogeneity, and (v) Energy.

Contrast is a measure of the difference in brightness between neighboring pixels in an image. A high contrast value indicates that adjacent pixels have significantly different brightness levels. Contrast is an important feature because it can differentiate between objects of different temperatures, which is crucial for accurately detecting and counting occupants. *Correlation* measures similarity between neighboring pixels in an image. A high correlation value indicates that adjacent pixels have similar brightness levels. Correlation can help to identify regions in an image with uniform textures, which may be useful for segmenting the image and separating different objects or regions. *Entropy* is a measure of the randomness or disorder in an image. A high entropy value indicates that the texture in an image is irregular or unpredictable. Entropy can be used to identify areas in an image that are likely to contain noise. *Homogeneity* measures the uniformity of textures in an image. A high homogeneity value indicates that adjacent pixels have similar brightness levels, which can help to identify regions in an image with uniform textures. *Energy* is a measure of the overall intensity or magnitude of textures in an image. A high energy value indicates that an image contains many high-intensity or high-magnitude textures. Mathematically,

$$\text{Contrast} = \sum_{i,j} |i - j|^2 p_{i,j}; \quad \text{Energy} = \sum_{i,j} p_{i,j}^2$$

$$\text{Entropy} = - \sum_{i,j} p_{i,j} \log_2 p_{i,j}; \quad \text{Homogeneity} = \sum_{i,j} \frac{p_{i,j}}{1 + |i - j|}$$

$$\text{Correlation} = \sum_{i,j} \frac{(i - \mu_i)(i - \mu_j)p_{i,j}}{\sigma_i \sigma_j}$$

Where the $p_{i,j}$ are the pixel values in the normalized co-occurrence matrix and μ_i quantity is an average calculated along rows of GLCM matrices, and μ_j is the average calculated along the columns. Similarly, σ_i and σ_j are standard deviations calculated along rows and columns.

Classification in ABFA*: ABFA*'s computed features are fed to ML-based classification algorithms for occupancy estimation as shown in Figure 3. We use two well-known algorithms for occupancy estimation, i.e., (i) Random Forest (RF) (ii) Neural network (NN). Following is a description of their parameter settings.

i. Random Forest (RF): RF is an ensemble learning algorithm that combines the predictions of multiple decision trees to make accurate predictions. These are constructed by creating a large number of randomized decision trees. Unlike a single decision tree, which are prone to overfitting or being overly sensitive to the training data, RF reduce these issues by introducing randomness in the construction process.

For evaluating ABFA* using RF, we used 100 decision trees to balance accuracy and computational cost. The depth of each tree was restricted to 10, to prevent over-fitting and enhance generalization, ensuring the trees remain manageable and not overly complex.

ii. Neural Network (NN): Figure 6 shows the NN model's architecture comprising multiple hidden layers with varying numbers of neurons. All the hidden layers use the Rectified Linear Unit (ReLU) activation function. Before defining the NN, we computed and normalized the mutual information scores between each feature and the ground truth label. These scores serve as weights for the input layer, prioritizing more informative features for the classification task. The NN takes computed features as input.

The output layer contains 16 neurons, corresponding to the number of classes (occupancy levels 0 to 15) in the classification task, and utilizes a *Softmax* activation function. The Adam optimizer trains the model using 'sparse categorical cross entropy' as the loss function and 'accuracy' as the evaluation metric. The chosen

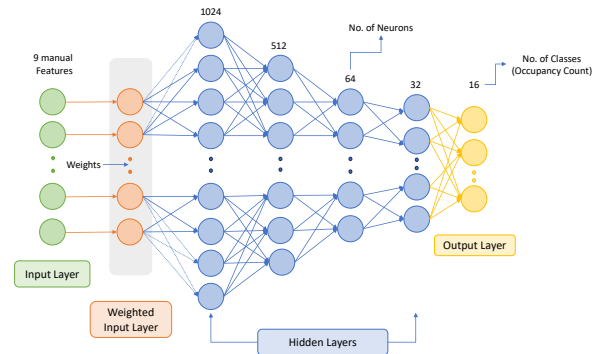


Figure 6: Architecture of Neural Network (NN)

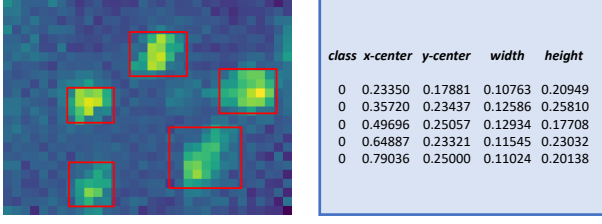


Figure 7: Left: Five annotated bounding boxes; right: Label file created after annotation.

architectural elements collectively aim to optimize accuracy for occupancy prediction, resulting in an effective and well-suited neural network for the specific task.

4.1.2 You Only Look Once (YOLO). It is an advanced object detection architecture that efficiently detects multiple objects in one pass by processing the entire image as a whole [15]. It uses a backbone convolutional NN to capture features from the input image, which are then passed through additional convolutional layers to predict bounding boxes and class probabilities for objects in the image.

We utilized the **YOLOv5s** model, a smaller variant of YOLOv5, for occupancy estimation and localization due to its faster inference speed, lower computation power, and memory requirements. Its lightweight design aligns well with energy optimization goals for edge devices. The model is primarily used for classification and detection tasks, but it also provides object localization by assigning confidence scores to predicted bounding boxes for target objects [15]. Since our task involves detecting occupants and counting the total number of bounding boxes in a thermal image, we focused on training the model for object detection rather than classification. In YOLO, the occupancy count refers to the number of detected bounding boxes in an image frame, and whenever the number of bounding boxes in the input image and the predicted image is same we categorize that as a 'correct match'. So

$$\text{Accuracy}(\%) = \frac{\text{No. of images with correct match}}{\text{Total no. of images in test dataset}} \times 100$$

The model has limitations in detecting small heat signatures, leading to false negatives (FN) and false positives (FP). Also, it may misclassify small thermal noise as an occupant. Consequently, this can lower the several metrics, calculated as follows.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \text{ Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{Intersection over Union (IoU)} = \frac{\text{Area of Overlap}}{\text{Total area of Union}} = \frac{A \cap B}{A \cup B}$$

Where TP represents true positive; A and B represent areas of predicted and actual bounding boxes, respectively. Ideally, values of Precision, Recall, and IoU should be one.

The annotation process for training a YOLO model involves manually labeling each object (occupant) of interest within an image. This typically involves drawing bounding boxes around the occupant and assigning a label to each box. We used a commonly

available 'Labellmg'¹ tool for the annotation process. The annotated images train the YOLO model to detect and locate occupants within the new thermal images. Figures 7 (Left) and 7 (Right) represent five annotated bounding boxes and corresponding label entries, respectively. In the label file, "0" represents the object class (0 for our entire dataset), and the remaining four values denote the x-center, y-center, width, and height of the bounding box.

4.2 Localization with YOLO

Our implementation retrieves thermal image frames from the camera and converts the single channel grayscale images to 3-channel RGB image format. The RGB images were set to 256×192 , maintaining the original aspect ratio of the grayscale images at 32×24 resolution. Then it uses the YOLOv5s to detect and obtain the bounding box coordinates of the occupants.

The number of occupants (O) refers to the count of detected bounding boxes. For each detected occupant, our method first obtains center coordinates (x, y) of the bounding box. Next, it uses a *MAP* function to map bounding-box co-ordinates to a physical zone. The *MAP* function defined in Algorithm 1 is explained as follows.

Algorithm 1 Function mapping an occupant to a zone

```

1: function MAP( $x, y, X\_Div, Y\_Div$ )
2:    $Zone\_Length \leftarrow \frac{1}{X\_Div}$ 
3:    $Zone\_Width \leftarrow \frac{1}{Y\_Div}$ 
4:    $Z\_x \leftarrow \lfloor x / Zone\_Length \rfloor + 1$ 
5:    $Z\_y \leftarrow \lfloor y / Zone\_Width \rfloor + 1$ 
6:   return Occupant's zone: ( $Z\_x, Z\_y$ )

```

Algorithm 2 Localization-cum Occupancy Estimation Algorithm

```

1: Input: Thermal image frames,  $F = [f_1, f_2, \dots, f_n]$ 
2: Input:  $X\_Div, Y\_Div$ 
3:  $ZoneMap \leftarrow []$ 
4: for  $i = 1$  to  $n$  do
5:   Convert  $f_i$  from grayscale to RGB
6:    $O \leftarrow$  Count occupants & coordinates in  $f_i$  with YOLO5
7:   for  $j = 1$  to  $O$  do
8:      $x, y \leftarrow$  center bounding box coordinates of  $O_j$ 
9:      $Z\_x, Z\_y \leftarrow \text{MAP}(x, y, X\_Div, Y\_Div)$ 
10:     $ZoneMap[i, j] \leftarrow (Z\_x, Z\_y)$ 
11:   end for
12: end for
13: return  $ZoneMap$ 

```

MAP function: It takes the parameters x, y, X_Div, Y_Div as input and returns the zone in which an occupant is present. Parameters, x, y refer to bounding box center coordinates while X_Div and Y_Div refer to the number of divisions along the x-axis (i.e., room length) and y-axis (i.e., room width) of an indoor space (i.e., captured area). The function divides physical space into zones based on the input parameters and maps bounding box coordinates to the zones. The

¹<https://pypi.org/project/labellmg/>

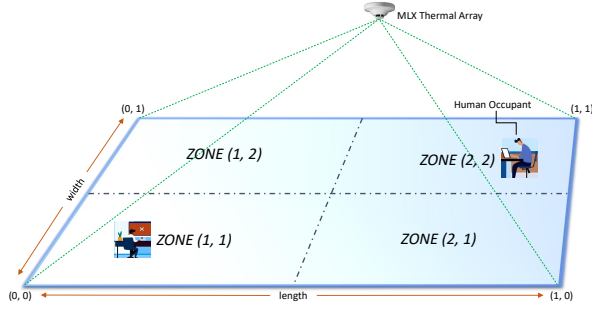


Figure 8: Two occupants in a four zone lab space.

function works as (i) Firstly, it calculates $Zone_Length$ by dividing the total frame length (i.e., 1 unit) by the number of divisions along the x-axis. Similarly, it calculates $Zone_Width$ by dividing the total frame width (i.e., 1 unit) by the number of divisions along the y-axis. (ii) Secondly, it determines the zone (Z_x) along the x-axis by dividing the x-coordinate, x , of the bounding box center by the zone length, $Zone_Length$, and taking the floor of the result. Adding one ensures that zone indices start from 1 instead of 0. Similarly, it determines the zone (Z_y) along the y-axis using the y-coordinate, y , of the bounding box center and the zone width, $Zone_Width$. (iii) Finally, it returns occupant's zone as tuple, (Z_x, Z_y).

Let's understand the occupancy estimation and localization with Figure 8. It shows a physical lab space divided into four zones and two occupants present in Zones (1, 1) and (2, 2). To estimate the mentioned zones, the proposed method defined in Algorithm 2 first converts the grayscale thermal frame f_i into RGB and then counts the 'number of occupants' ($O = 2$) in the frame with YOLO5. Next, for each occupant (o_1, o_2), it computes their center box coordinates, (x_1, y_1) and (x_2, y_2) as (0.2, 0.2) and (0.8, 0.7), respectively. Next, it uses *Map* function (Algorithm 1) and computes $Zone_Length = 1/2 = 0.5$, $Zone_Width = 1/2 = 0.5$. Finally, it estimates occupant # 1's zone as: $Z_x = \lfloor 0.2/0.5 \rfloor + 1 = 0 + 1 = 1$, $Z_y = \lfloor 0.2/0.5 \rfloor + 1 = 0 + 1 = 1$. Similarly, occupant # 2's zone is: $Z_x = \lfloor 0.8/0.5 \rfloor + 1 = 1 + 1 = 2$, $Z_y = \lfloor 0.7/0.5 \rfloor + 1 = 1 + 1 = 2$. The example shows that the proposed method estimated zones for occupants o_1 and o_2 as (1, 1) and (2, 2) successfully.

4.3 U-Net: Baseline for occupancy estimation

We selected the U-Net Convolutional Neural Network (CNN)-based image segmentation model as the baseline for several compelling reasons. Firstly, U-Net offers the advantage of a publicly available dataset and a readily accessible GitHub² repository, simplifying its implementation and ensuring transparency. Secondly, the choice of U-Net was motivated by key shared parameters, such as the utilization of the same thermal camera (MLX90640) with a resolution of 32×24 pixels in our experiments and a prior study ([17]). This commonality facilitates a meaningful comparative analysis between our proposed model and U-Net. Furthermore, the consistent experimental settings, such as the positioning of the thermal sensor

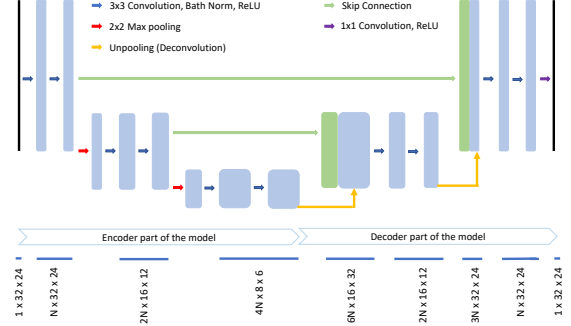


Figure 9: Architecture of U-Net

on the ceiling, further reinforce our rationale for using U-Net as the baseline model, ensuring a robust foundation for our study.

Figure 9 illustrates the U-Net model architecture, depicting the connections between each layer and their respective shapes. It comprises of two parts: the encoder and the decoder. We use single-channel images with a resolution of 32×24 as inputs to simplify the network while maintaining its performance, making it suitable for embedded devices. We followed [17] while reducing the number of layers in the encoder and decoder stages and decreasing the number of filters. Our network operates with $N = 32$, where N represents the number of kernels (filters) applied to the input image.

To generate the ground truth from INFRADEL, we start with an empty image with the same dimensions as the input thermal image. We mark the locations of occupants in this empty image by clicking on these points with a mouse, as shown in the lower left part in Figure 10. Then, we set a single pixel in the ground truth image to full brightness, corresponding to each marked location.

To create the density map, a fixed 2D Gaussian mask is convolved with the ground truth image. This process generates a Gaussian mixture distribution where the maximums (single pixels set to full brightness) represent the occupants' locations. The sigma value for the annotation is determined using a grid-search technique to achieve a blob size approximately equal to that of a single Gaussian mask. As the camera's height was consistent in all scenarios, the Gaussian parameter, $\sigma = 1.75$, remained constant throughout.

The figure shows the Gaussian density map created by convolution, where the sum of pixel values for an occupant is approximately 18.75 with $\sigma = 1.75$. We calculate the total sum of pixels in the reconstructed image to determine the number of people in a frame from the model's output (Figure 11). Dividing this sum by the sum of pixels corresponding to a single occupant (18.75) gives the occupancy count. In this case, the sum of pixels in the reconstructed image was 149.44, leading to an occupancy count of 8 (rounded from 7.97).

$$\text{Number of occupants} = \frac{\text{Sum of pixels in the model output}}{\text{Sum of pixels for a single occupant}}$$

²<https://github.com/PUTvision/thermo-presence>

Table 1: List of different features

Abbreviation	Feature(s)
F1	mean (m)
F2	std. deviation (σ)
F3	number of active pixels
F4	size of 3 largest blobs [8]
F5	total number of connected components
F6	size of the biggest blob [5]
F7	skewness
F8	kurtosis
F9	energy
F10	homogeneity
F11	contrast
F12	correlation
F13	entropy

Table 2: Feature combinations

Combination	Features
C1	F1, F2, F3
C2	F1, F2, F3, F4
C3	F1, F2, F3, F5
C4	F1, F2, F3, F6
C5	F1, F2, F3, F7, F8
C6	F1, F2, F3, F4, F7, F8
C7	F1, F2, F3, F6, F7, F8
C8	F1, F2, F3, F5, F7, F8
C9	F1, F2, F3, F5, F7, F8, F9, F10, F11
C10	F1, F2, F3, F5, F7, F8, F9, F10, F12
C11	F1, F2, F3, F5, F7, F8, F9, F11, F13
C12	F1, F2, F3, F5, F7, F8, F10, F12, F13
C13	F1, F2, F3, F5, F7, F8, F11, F12, F13

Table 3: Performance of ABFA* (RF) with different feature combinations under dense settings on our INFRADEL test dataset.

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13
F1-score	0.79	0.79	0.81	0.80	0.82	0.90	0.91	0.94	0.95	0.95	0.94	0.93	0.96
Accuracy (%)	78.88	80.37	80.98	79.26	82.59	89.69	92.91	94.44	94.88	95.03	93.33	92.69	96.33

5 RESULTS

In this section, first, we present the occupancy estimation results and, later, the sensitivity analysis of YOLO. During occupancy estimation, we thoroughly analyze the feature importance and selection process, exploring various feature combinations and ultimately identifying the top nine features. The results were obtained on the test dataset where the dataset was divided into training, validation, and testing sets as 70%, 20%, and 10%, respectively.

5.1 Occupancy Estimation

Table 1 lists computed manual features, while Table 2 shows different combinations $C_i | i \in \{1, \dots, 13\}$ of these features. Table 3 presents occupancy estimation results (F1-score, Accuracy) with ABFA* (with RF) in dense settings. For instance, combination C1 comprising of features mean (F1), standard deviation (F2), and number of active pixels (F3) achieves 78.88% accuracy. C3, which adds the number of connected components as a feature, outperforms the combination with the size of the three largest blobs (which require manual tuning). The inclusion of all first-level parameters in C8 results in better performance using F5 instead of F4 and F6. So, we select F5 in addition to first-level features in remaining feature combinations. C13, combining all first-level and three second-level features with the number of connected components, provides the best performance. So, we used C13 feature combination in all the ML-based occupancy estimation algorithms.

Table 4 shows the occupancy estimation accuracy of ABFA, ABFA*, U-Net, and YOLO for different occupancy levels. Note that ABFA and ABFA* used C3, and C13 feature combinations, respectively, while deep models (U-Net, YOLOv5s) extracted features autonomously from the raw data. ABFA*, trained on RF, demonstrates an impressive accuracy of 96%, surpassing the performance

of the ABFA algorithm (85%) by a substantial margin of 10%. Notably, ABFA* overcomes the limitations of ABFA by employing an advanced set of feature vectors, which significantly mitigates the rapid decrease in performance observed after occupancy level 11. Overall, YOLOv5s, a deep-learning occupancy estimation method, performs best.

To test the performance of YOLO, we used visualizations of the model output on a sample dynamic video to assess the accuracy of the bounding box predictions. The model was able to detect occupants in various scenarios accurately. Figure 12 (right) demonstrate

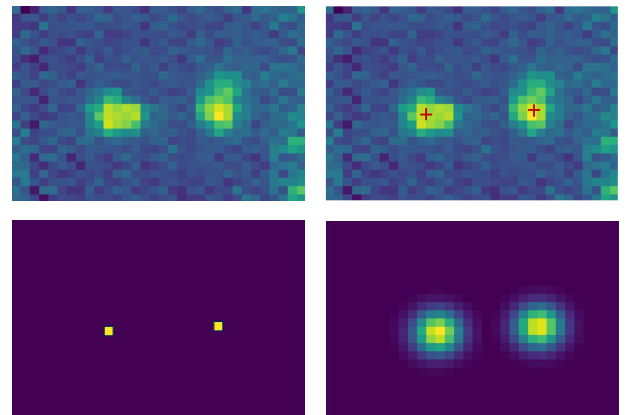
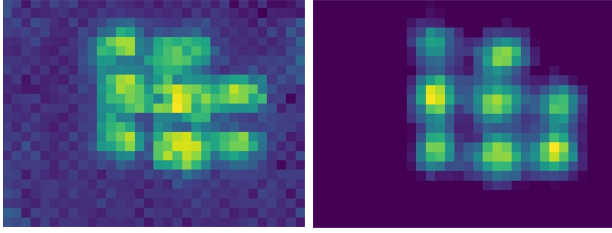


Figure 10: Upper left: a thermal image of the occupants; upper right: image with a mouse click at the actual location of the two persons; lower left: two pixels set to the maximum brightness (255) at marked location; lower right: generated density maps around the occupants' locations

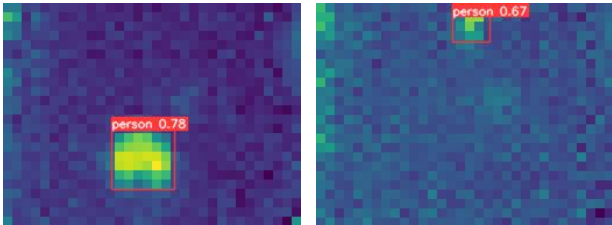
Table 4: Occupancy estimation accuracy (%) of ABFA, ABFA*, U-Net, and YOLOv5s with varying occupancy (1 - 15) on our INFRADEL dataset.

Occupancy (#)	ABFA (RF)	ABFA* (RF)	ABFA* (NN)	U-Net	YOLOv5s
1	99.98	100	99.69	87.64	100
2	99.53	99.53	99.57	98.50	88.89
3	98.01	98.99	97.44	100	94.44
4	95.91	99.44	97.21	99	100
5	95.07	97.27	95.49	60.29	96.15
6	94.36	99.08	93.86	87	98.44
7	94.67	96.70	92.24	98	98.36
8	94.39	96.56	87.02	93.18	100
9	93.79	98.35	95.41	82.50	99.17
10	92.84	97.94	99.54	95.48	99.09
11	91.33	97.64	85.36	87	98.08
12	86.73	95.89	80.23	99.50	96.67
13	61.41	98.05	74.19	93.37	98.32
14	53.57	96.82	88.13	97	98.61
15	49.24	99.82	89.27	52.44	97.1
Overall	84.50	96.33	89.67	89.42	97.68

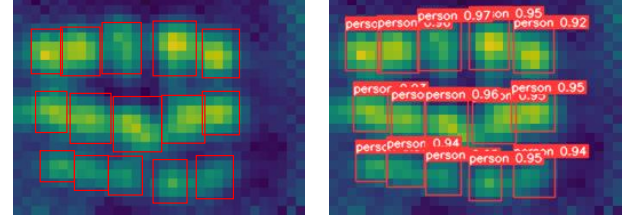
**Figure 11: Left: input thermal image; right: reconstructed thermal image (model output)**

the model’s effectiveness, where it accurately predicts an edge case of an occupant standing near the boundary of the coverage zone with a confidence score of 0.67, indicating a satisfactory level of certainty. However, the model could not provide an exact occupancy count in cases of occlusion (i.e., the model output a single bounding box when two occupants were present very close to each other as shown in Figure 12). One must note that *this limitation of the model should not be interpreted in the case of a static scenario, as it occurred only when two occupants crossed paths with each other in a dynamic setting*.

Figure 14 displaying the confusion matrices of two deep learning models, U-Net and YOLO, reveals that the YOLO model consistently

**Figure 12: Left: occlusion case (two persons crossing each other predicted as one). Right: prediction in edge case with a confidence score of 0.67.****Table 5: Performance of different occupancy estimation models on the publicly available Thermo-presence dataset.**

	ABFA* (RF)	ABFA* (NN)	ABFA (RF)	U-Net	YOLOv5s
Accuracy (%)	95.14	89.17	92.83	91.72	93.67
Average IoU (%)	-	-	-	-	87.39

**Figure 13: Actual (left) and predicted (right) bounding boxes with YOLO.**

outperforms the U-Net model across almost all classes. Notably, the U-Net model exhibits a higher rate of false positive cases for classes 5, 9, 11, and 15. This occurrence can be due to numerous images within these classes where occupants are partially visible. The U-Net’s reliance on rounding off final prediction values to the nearest integers contributes to this behavior. For instance, if a partially detected occupant’s Gaussian model output contribution is < 0.5 towards the occupancy count, the U-Net’s final model output will be one count less than it should have been. In contrast, the YOLO model correctly classifies almost all the images with a very few mis-classifications, which were later discovered to be the images collected in a dynamic setting.

5.1.1 Occupancy estimation on Thermo-presence dataset. Besides the INFRADEL dataset, we evaluate the robustness of our models on a publicly available Thermo-presence³ dataset. Results in Table 5 show that the ABFA* exhibits remarkably high accuracy across both level 1 and level 2 feature sets, even without background images – a crucial component for threshold frame calculation.

In contrast, YOLOv5s also excels with the dataset despite being trained with a limited number of dynamic images. However, challenges arise in dynamic conditions where occupants move, leading to complex occlusion scenarios and residual heat signatures. This complexity poses difficulties in distinguishing between actual human heat patterns and those left behind by dynamic signatures, resulting in a 5 to 7% decrease in the bounding box accuracy of YOLOv5s. This noteworthy achievement lies in the impressive performance of our proposed models on the dataset, affirming their generalizability across various temporal conditions.

6 PERFORMANCE ON AN EDGE DEVICE

6.1 Sensitivity Analysis of YOLO

Table 6 displays the performance results of the YOLOv5s model with various parameter settings, including image size, model freezing, optimizer, and learning rate, as well as transfer learning. The

³<https://github.com/PUTvision/thermo-presence>

evaluated performance metrics include mean average precision (mAP), box loss, mAP@0.5, mAP@0.5:0.95, and object loss, which are defined as follows: (i) *Box loss*: Measures the loss incurred in the predicted bounding boxes, indicating how far off the predicted bounding boxes are from the ground truth boxes. (ii) *Objectness loss*: Represents the loss incurred in the objectness score of the predicted bounding boxes, where the objectness score assesses the likelihood of an object being present in the bounding box. (iii) *mAP@0.5*: Represents the mean Average Precision (mAP) of the model at a confidence threshold of 0.5. A higher mAP value indicates accurate object detection in the images. (iv) *mAP@0.5:0.95*: Calculates the mean average precision averaged over multiple IoU thresholds ranging from 0.5 to 0.95. This provides a comprehensive assessment of the model's accuracy across different IoU thresholds.

The evaluation demonstrated that the YOLOv5s model achieved high precision, recall, and mAP scores for both 256 and 640 image sizes. The larger image size yielded slightly better mAP scores and box loss, indicating improved accuracy. However, due to computation limitations, the smaller image size was chosen for training. The model converged within approximately 200 epochs and performed consistently best with the Adam optimizer.

We also examined the impact of model freezing by freezing different numbers of layers. Freezing the first ten layers decreased performance compared to the unfrozen model, as indicated by slightly lower precision (0.976) and recall (0.967) scores. However, the mAP score decreased significantly (0.551), suggesting a notable decline in accuracy. Freezing the first 15 layers had a similar effect, further decreasing precision (0.978) and recall (0.963), as well as the map score (0.525).

We explored the potential benefits of transfer learning by evaluating the Nano model (YOLOv5n), a lightweight model variant. The model yielded impressive results with high precision (0.998), recall (0.997), and mAP scores (0.994). The corresponding box loss (0.025) and object loss (0.020) were also favorable. Although the required IoU score metric remained relatively unchanged, this still indicates that the Nano model can achieve accurate object detection while maintaining a relatively small model size.

We also explored optimization techniques by testing different optimizers. The Stochastic Gradient Descent (SGD) optimizer performed well, with high precision (0.999), recall (0.998), and mAP scores (0.994). However, the improvement in the desired IoU metric was not significant. Further optimization was achieved by changing

Table 6: Training results for various parameter configurations. Common parameters include: Epochs - 200, Batch size - 32, Image size - 256 x 256, and optimizer - Adam

Parameter	precision	recall	mAP_0.5	mAP_0.5:0.95	box_loss	obj_loss
img_size_256	0.999	0.997	0.994	0.867	0.028	0.023
img_size_640	0.998	0.996	0.994	0.884	0.017	0.019
frozen_10	0.976	0.967	0.988	0.551	0.046	0.043
frozen_15	0.978	0.963	0.988	0.525	0.048	0.045
Nano_model	0.998	0.997	0.994	0.884	0.025	0.020
opt_SGD	0.999	0.998	0.994	0.910	0.022	0.019
lr_0.005	0.999	0.999	0.995	0.926	0.022	0.017
Best*	0.999	0.999	0.996	0.933	0.021	0.015

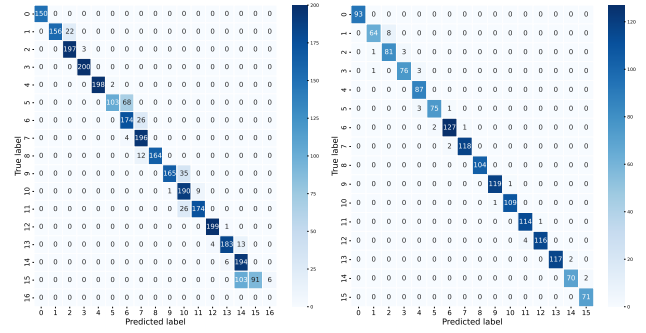


Figure 14: Confusion matrices of U-Net (left) and YOLO (right).

the learning rate from 0.01 to 0.005, which improved the IoU score and enhanced stability in the loss metric.

After multiple iterations, we identified the **Best*** values for various parameters. These parameters include an image size of 256×256, a learning rate of 0.001, and the Adam optimizer for retraining the YOLOv5s model. The lower learning rate (0.001) effected the convergence time of the model and required 300 epochs for model training to achieve stability for loss value. The final training and validation results demonstrated a box loss of approximately 0.021, an obj loss of 0.015, a mean average precision (map_0.5) of 0.996, and a mAP_0.5:0.95 of 0.933. Figure 13 illustrates the input and output thermal images with accurately predicted bounding boxes around occupants. These results were obtained using the YOLO model trained with the optimized parameters from the **Best*** configuration.

We evaluated our models' (ABFA*, U-Net, YOLO) real-time performance on a Raspberry Pi (RPI), the same device used during data collection. The evaluation included metrics such as latency (time delay between input and output during inference), average inference time (typical duration for one inference), CPU usage (proportion of the CPU utilized during inference), & device temperature (heat generated during model execution). Monitoring & managing the device's temperature are crucial for ensuring optimal performance & preventing overheating issues. Table 7 shows the evaluation results.

Our proposed ABFA* (with RF) demonstrated the highest accuracy among the ML models, achieving 96% accuracy, surpassing the previous models trained on the Blob Filtering Algorithm (BFA) [8]. The model, in a converted ".pkl" format, was loaded onto the RPi to analyze live-captured thermal images. The measured inference time was 20 ms, with an average latency of 1.02 seconds per image. The increased latency is primarily attributed to the feature extraction process, which consumes a significant amount of time.

The ABFA* (with NN) trained on the same manual features as the ABFA* (RF) achieved an accuracy of 87% on the RPi. To reduce its computational load, the TensorFlow model was converted to a lighter version called "TensorFlow Lite," which was then saved and loaded onto the device for evaluation. The TensorFlow Lite model demonstrated faster predictions, taking only 6 ms per image. However, similar to the RF model, the NN also exhibited higher latency due to the consistent feature extraction process.

Table 7: Performance of models on Raspberry Pi (Model 4B)

	Accuracy (%)	Latency (s)	Inf. time (s)	CPU usage (%)	Temp (°C)
ABFA*(RF)	96.33	1.02	0.020	35	60
ABFA*(NN)	87.88	0.98	0.006	31	61
BFA (RF) [8]	92.37	0.87	0.015	33	58
U-Net [17]	87.08	0.69	0.025	21	52
YOLOv5s	97.16	1.25	0.250	25	55

Among DL models, YOLO demonstrates higher accuracy, but it incurs longer inference time and latency. This is due to the architectural complexity of the model and the requirement of a 3-channel RGB input for the YOLOv5s model, which increases overall latency. There are other Deep Learning models that consume less power and exhibit significantly faster performance than YOLOv5s. These models use a 2-dimensional image array as input and have smaller model sizes. However, they are more suitable for detection and estimation tasks in sparse settings with limited number of occupants.

Table 7 shows that the DL-based models (U-Net, YOLO) utilize less CPU and generate less heat compared to ML-based models. However, the optimal choice among these models depends on the specific deployment scenario and the trade-offs involved. For instance, the NN model proves advantageous in static sparse scenarios, offering higher accuracy and lower inference time, though it may suffer from reduced accuracy in dense settings. On the other hand, the RF model exhibits versatility, performing well in both dense and sparse scenarios, making it suitable for applications like classrooms with diverse settings.

In conclusion, our results indicate that the proposed models can achieve high accuracy on low-power edge devices, making them well-suited for real-time deployment in various applications. These models strike a balance between accuracy and computational efficiency, making them practical and effective solutions for real-world scenarios.

7 CONCLUSION AND FUTURE WORK

Our work significantly advances occupancy estimation and localization techniques, focusing on smart buildings. We proposed and rigorously evaluated ML and DL-based methods tailored for both dense and sparse occupancy settings, encompassing both static and dynamic scenarios. The comprehensive evaluation of these methods across various occupancy settings ensures their suitability for real-world applications. Including both ML and DL models provides valuable flexibility, allowing users to choose the most suitable method for their specific use cases. Moreover, our occupant localization method, which utilizes YOLO, offers adaptability in diverse room settings, facilitating efficient control of devices such as LEDs, fans, and more.

The presented models exhibit exceptional accuracy when compared to state-of-the-art approaches. However, there is still room for improvement in terms of latency issues. For ML-based models, conducting a more intricate evaluation to select optimal features & assessing their performance on edge devices could lead to faster response times, even though some accuracy might be sacrificed by choosing a reduced number of features. Similarly, for DL-based models, exploring quicker methods to convert thermal images from grayscale to RGB format & optimizing loading processes could

significantly improve latency. Additionally, investigating model quantization may enhance real-world deployment scenarios.

Our results lay a robust foundation for future enhancements in latency reduction and improved model deployment, bringing us closer to creating more efficient and effective systems for building management. With continued research and innovation, we aim to achieve even greater efficiency, facilitating the seamless integration of occupancy estimation and localization techniques into smart buildings and beyond.

ACKNOWLEDGMENTS

We thank our fellow lab members and friends for supporting and providing valuable insights into this work. We also thank the Classroom support team for their assistance in data collection.

REFERENCES

- [1] Jawad Ahmad, Hadi Larjani, R Emmanuel, M Mannion, and A Javed. 295. Occupancy detection in non-residential buildings—A survey and novel privacy preserved occupancy monitoring solution. *Applied Computing and Informatics* 17, 2 (295), 279.
- [2] Jack Andrews, Meghana Kowsika, Asad Vakil, and Jia Li. 2020. A Motion Induced Passive Infrared (PIR) Sensor for Stationary Human Occupancy Detection. In *2020 IEEE/ION Position, Location and Navigation Symposium (PLANS)*. IEEE, Portland, Oregon, 1295–1304. <https://doi.org/10.1109/PLANS46316.2020.9109909>
- [3] Omid Ardakanian, Arka Bhattacharya, and David Culler. 2018. Non-intrusive occupancy monitoring for energy conservation in commercial buildings. *Energy and Buildings* 179 (2018), 311–323.
- [4] Laleh Armi and Shervan Fekri-Ershad. 2019. Texture image analysis and texture classification methods - A review. *arXiv:1904.06554 [cs.CV]* <https://arxiv.org/abs/1904.06554>
- [5] Chandrayee Basu and Anthony Rowe. 2015. Tracking Motion and Proxemics using Thermal-sensor Array. *CoRR* abs/1511.08166 (2015), N/A. [arXiv:1511.08166](https://arxiv.org/abs/1511.08166)
- [6] Alex Beltran and Alberto E. Cerpa. 2014. Optimal HVAC building control with occupancy prediction. In *Proceedings of the 1st ACM Conference on Embedded Systems for Energy-Efficient Buildings (Memphis, Tennessee) (BuildSys '14)*. Association for Computing Machinery, New York, NY, USA, 168–171. <https://doi.org/10.1145/2674061.2674072>
- [7] Mondher Bouazizi, Chen Ye, and Tomoaki Ohtsuki. 2022. Low-resolution infrared array sensor for counting and localizing people indoors: When low end technology meets cutting edge deep learning techniques. *Information* 13, 3 (2022), 132.
- [8] Veena Chidurala and Xinrong Li. 2021. Occupancy estimation using thermal imaging sensors and machine learning algorithms. *IEEE Sensors Journal* 21, 6 (2021), 8627–8638.
- [9] Veena Chidurala and Xinrong Li. 2022. Detection of moving objects using thermal imaging sensors for occupancy estimation. *Internet of Things* 17 (2022), 100487.
- [10] Mertcan Cokbas, Prakash Ishwar, and Janusz Konrad. 2020. Low-Resolution Overhead Thermal Tripwire for Occupancy Estimation. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. IEEE, Seattle, WA, USA, 398–406. <https://doi.org/10.1109/CVPRW50498.2020.00052>
- [11] Aweek K. Das, Parth H. Pathak, Josiah Jee, Chen-Nee Chuah, and Prasant Mohapatra. 2017. Non-Intrusive Multi-Modal Estimation of Building Occupancy. In *Proceedings of the 15th ACM Conference on Embedded Network Sensor Systems (Delft, Netherlands) (SenSys '17)*. Association for Computing Machinery, New York, NY, USA, Article 14, 14 pages. <https://doi.org/10.1145/3131672.3131680>
- [12] Varick L. Erickson, Miguel Á. Carreira-Perpiñán, and Alberto E. Cerpa. 2011. OBSERVE: Occupancy-based system for efficient reduction of HVAC energy. In *Proceedings of the 10th ACM/IEEE International Conference on Information Processing in Sensor Networks*. IEEE, Chicago, IL, USA, 258–269.
- [13] Robert M. Haralick, K. Shanmugam, and Its'hak Dinstein. 1973. Textural Features for Image Classification. *IEEE Transactions on Systems, Man, and Cybernetics* SMC-3, 6 (Nov 1973), 610–621. <https://doi.org/10.1109/TSMC.1973.4309314>
- [14] Shushan Hu, Peng Wang, Cathal Hoare, and James O'Donnell. 2022. Building Occupancy Detection and Localization Using CCTV Camera and Deep Learning. *IEEE Internet of Things Journal* 10, 1 (2022), 597–608.
- [15] Marina Ivašić-Kos, Mate Krišto, and Miran Pobar. 2019. Human Detection in Thermal Imaging Using YOLO. In *Proceedings of the 2019 5th International Conference on Computer and Technology Applications (Istanbul, Turkey) (IC-CTA '19)*. Association for Computing Machinery, New York, NY, USA, 20–24. <https://doi.org/10.1145/3323933.3324076>

- [16] Ming Jin, Nikolaos Bekiaris-Liberis, Kevin Weekly, Costas J. Spanos, and Alexandre M. Bayen. 2018. Occupancy Detection via Environmental Sensing. *IEEE Transactions on Automation Science and Engineering* 15, 2 (2018), 443–455. <https://doi.org/10.1109/TASE.2016.2619720>
- [17] Marek Kraft, Przemysław Aszkowski, Dominik Pieczyński, and Michał Fularz. 2021. Low-Cost Thermal Camera-Based Counting Occupancy Meter Facilitating Energy Saving in Smart Buildings. *Energies* 14, 15 (2021), 4542.
- [18] Song Liu and Tian He. 2017. SmartLight: Light-weight 3D Indoor Localization Using a Single LED Lamp. In *Proceedings of the 15th ACM Conference on Embedded Network Sensor Systems (Delft, Netherlands) (SenSys '17)*. Association for Computing Machinery, New York, NY, USA, Article 11, 14 pages. <https://doi.org/10.1145/3131672.3131677>
- [19] Aly Metwaly, Jorge Peña Queralta, Victor Kathan Sarker, Tuan Nguyen Gia, Omar Nasir, and Tomi Westerlund. 2020. Edge Computing with Embedded AI: Thermal Image Analysis for Occupancy Estimation in Intelligent Buildings. In *Proceedings of the INTelligent Embedded Systems Architectures and Applications Workshop 2019 (New York, NY, USA) (INTESA2019)*. Association for Computing Machinery, New York, NY, USA, 1–6. <https://doi.org/10.1145/3372394.3372397>
- [20] V. Nivetha, B. Subathra, and Seshadri Srinivasan. 2019. Wi-Fi based Occupancy Detection in a Building with Indoor Localization. In *2019 IEEE International Conference on Intelligent Techniques in Control, Optimization and Signal Processing (INCOS)*. IEEE, Tamilnadu, India, 1–4. <https://doi.org/10.1109/INCOS45849.2019.8951381>
- [21] Zishan Qin, Dipankar Chaki, Abdallah Lakhdari, Amani Abusafia, and Athman Bouguettaya. 2021. Occupancy Estimation from Thermal Images. *CoRR* abs/2110.07796 (2021), 301–305. arXiv:2110.07796 <https://arxiv.org/abs/2110.07796>
- [22] Hamid Rajabi, Zhizhang Hu, Xianzhong Ding, Shijia Pan, Wan Du, and Alberto Cerpa. 2022. MODES: Multi-sensor occupancy data-driven estimation system for smart buildings. In *Proceedings of the Thirteenth ACM International Conference on Future Energy Systems (Virtual Event) (e-Energy '22)*. Association for Computing Machinery, New York, NY, USA, 228–239. <https://doi.org/10.1145/3538637.3538852>
- [23] Mikko Rinta-Homi, Naser Hossein Motlagh, Agustín Zuniga, Huber Flores, and Petteri Nurmi. 2021. How Low Can You Go? Performance Trade-offs in Low-Resolution Thermal Sensors for Occupancy Detection: A Systematic Evaluation. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 5, 3 (2021), 1–22.
- [24] Ali Saffari, Sin Yong Tan, Mohamad Katanbaf, Homagni Saha, Joshua R. Smith, and Soumik Sarkar. 2021. Battery-Free Camera Occupancy Detection System. In *Proceedings of the 5th International Workshop on Embedded and Mobile Deep Learning (Virtual, WI, USA) (EMDL'21)*. Association for Computing Machinery, New York, NY, USA, 13–18. <https://doi.org/10.1145/3469116.3470013>
- [25] Soumya R Sahoo and Haroon R Lone. 2023. Occupancy counting in dense and sparse settings with a low-cost thermal camera. In *2023 15th International Conference on COMMunication Systems & NETWORKS (COMSNETS)*. IEEE, Bangalore, India, 537–544.
- [26] ARPA-E SENSOR. 2017. Saving energy nationwide in structures with occupancy recognition. *US Department of Energy, Advanced Research Projects Agency Energy* N/A, N/A (2017), N/A.
- [27] Pragya Sharma, Guoyi Xu, Xiaonan Hui, David Lee Hysell, and Edwin Chihchuan Kan. 2021. Deep-Learning-Based Occupant Counting by Ambient RF Sensing. *IEEE Sensors Journal* 21, 6 (2021), 8564–8574. <https://doi.org/10.1109/JSEN.2020.3045035>
- [28] Oliver Shih and Anthony Rowe. 2015. Occupancy estimation using ultrasonic chirps. In *Proceedings of the ACM/IEEE Sixth International Conference on Cyber-Physical Systems (Seattle, Washington) (ICCPs '15)*. Association for Computing Machinery, New York, NY, USA, 149–158. <https://doi.org/10.1145/2735960.2735969>
- [29] Elah Soltanaghaei, Rahul Anand Sharma, Zehao Wang, Adarsh Chittilappilly, Anh Luong, Eric Giler, Katie Hall, Steve Elias, and Anthony Rowe. 2020. Robust and Practical WiFi Human Sensing Using On-device Learning with a Domain Adaptive Model. In *Proceedings of the 7th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation (Virtual Event, Japan) (BuildSys '20)*. Association for Computing Machinery, New York, NY, USA, 150–159. <https://doi.org/10.1145/3408308.3427983>
- [30] Tamim Sookoor and Kamin Whitehouse. 2013. RoomZoner: occupancy-based room-level zoning of a centralized HVAC system. In *Proceedings of the ACM/IEEE 4th International Conference on Cyber-Physical Systems (Philadelphia, Pennsylvania) (ICCPs '13)*. Association for Computing Machinery, New York, NY, USA, 209–218. <https://doi.org/10.1145/2502524.2502553>
- [31] Soumya Ranjan Sahoo, Haroon R Lone. 2024. INFRADEL Dataset. <https://osf.io/9u4yb/>. [Accessed: February 26, 2024].
- [32] Ash Tyndall, Rachel Cardell-Oliver, and Adrian Keating. 2016. Occupancy estimation using a low-pixel count thermal imager. *IEEE Sensors Journal* 16, 10 (2016), 3784–3791.
- [33] Chen Xie, Francesco Daghero, Yukai Chen, Marco Castellano, Luca Gandolfi, Andrea Calimera, Enrico Macii, Massimo Poncino, and Daniele Jahier Pagliari. 2023. Efficient Deep Learning Models for Privacy-Preserving People Counting on Low-Resolution Infrared Arrays. *IEEE Internet of Things Journal* 10, 15 (Aug 2023), 13895–13907. <https://doi.org/10.1109/JIOT.2023.3263290>
- [34] Yanbing Yang, Jie Hao, Jun Luo, and Sinno Jialin Pan. 2017. CeilingSee: Device-free occupancy inference through lighting infrastructure based LED sensing. In *2017 IEEE International Conference on Pervasive Computing and Communications (PerCom)*. IEEE, Kona, HI, USA, 247–256. <https://doi.org/10.1109/PERCOM.2017.7917871>
- [35] Liang Zhao, Yuxin Li, Ruobing Liang, and Peng Wang. 2022. A State of Art Review on Methodologies of Occupancy Estimating in Buildings from 2011 to 2021. *Electronics* 11, 19 (2022), 3173.
- [36] Han Zou, Yuxun Zhou, Jianfei Yang, and Costas J Spanos. 2018. Device-free occupancy detection and crowd counting in smart buildings with WiFi-enabled IoT. *Energy and Buildings* 174 (2018), 309–322.